

minuta

Minuta — Tema 07: Paradigma Lógico — Clase 2+3 (Unificación, Backtracking, Listas y Aplicaciones)

Materia: Paradigmas y Lenguajes de Programación 2026 **Docente:** Matías Gel — UNTDF / IDEI
Duración: 240 minutos | **Clase doble** **Fecha:** 2026-04-21

Uso: esta minuta es el guion **operativo** del docente. Tiempos reales, frases clave, momentos socráticos, checkpoints de pizarra. Se lee de corrido durante la clase.

Preparación previa (10 min antes de entrar)

- ☐ Abrir SWISH (<https://swish.swi-prolog.org/>) en una pestaña vacía
 - ☐ Preparar archivo `familia.pl` con la base de Tema 06 cargada
 - ☐ Abrir filminas en modo presentación
 - ☐ Tener pizarrón limpio + 2 colores de fibra
 - ☐ Copiar el código de B8–B11 a un snippet en el editor para live-coding rápido
-

BLOQUE 0 — Repaso (0–10 min) · F-001 a F-005

Objetivo pedagógico: reactivar memoria de Tema 06, anclar en el gancho “abrimos la caja negra”.

Guion

- **0–2 min · F-001:** saludo, anuncio de la clase doble. Frase apertura: *“Hoy vamos a abrir la caja negra de Prolog — qué pasa realmente cuando el motor responde.”*
- **2–5 min · F-002:** recorrer tabla de repaso; pedir 1 ejemplo de cada cosa a mano alzada.
- **5–8 min · F-003:** quiz relámpago (4 preguntas, sí/no, votación a mano).

- **8–10 min · F-004 + F-005:** mostrar lo que NO vimos y el mapa de 240 min. Anunciar descanso a los 120 min.

Checkpoint

“¿Alguien vio algo que **no** cerró de clase 1?” — si aparece unificación/backtracking → decir “justo, arrancamos con eso”.

BLOQUE 1 — Unificación (10–35 min) · F-006 a F-023

Objetivo: dominar el algoritmo de unificación y los 4 casos.

Guion

- **10–13 min · F-006, F-007:** definir unificación como *matching con pega*. Analogía: puzzle con variables.
- **13–18 min · F-008, F-009:** ejemplos de falla; subrayar que `=` NO evalúa (`X = 2+3` deja `X = 2+3`).
- **18–23 min · F-010, F-011:** escribir el algoritmo en pizarrón; trazar a mano `padre(juan, hijo(pedro, X)) = padre(Y, hijo(Z, ana))`.
- **23–26 min · F-012:** MGU. Dejar claro: “la sustitución más general que funcione”.
- **26–29 min · F-013, F-014:** unificación sobre hechos y reglas; mostrar en SWISH.
- **29–32 min · F-015, F-016:** `=` vs `==` vs `=..` — **éste** es el bloque donde se confunden siempre. Dedicar 2 min a insistir.
- **32–34 min · F-017, F-018:** occurs-check — ejemplo en vivo `?- X = f(X).` y mostrar el ciclo infinito.
- **34–35 min · F-019 + F-020:** `_` anónima + pattern matching. Cerrar con F-022, F-023 como auto-test.

Live-coding (SWISH)

```
?- padre(juan, maria) =.. L.
?- X = 2+3, Y is X.
?- X = Y, Y = 5, X.      % X queda Ligado a 5
?- X = f(X).             % occurs-check demo
```

Checkpoint

“¿Quién puede explicarme en voz alta por qué `?- f(X,X) = f(ana, beatriz).` falla?” — elegir alumno.

BLOQUE 2 — Resolución SLD (35–60 min) · F-024 a F-038

Objetivo: entender que una ejecución = una demostración formal.

Guion

- **35–40 min · F-024 a F-026:** definición SLD + vocabulario + algoritmo. Dejar el pseudocódigo en la pizarra toda la clase.
- **40–50 min · F-027 a F-030:** ejemplo integrado `abuelo(ana, N)` — dibujar el árbol SLD completo en pizarrón. Pedir participación: “¿qué cláusula unifica acá?”.
- **50–55 min · F-031, F-032:** regla de selección leftmost + contra-ejemplo del `ancestro` con orden malo.
- **55–58 min · F-033, F-034:** DFS + top-down.
- **58–60 min · F-035 a F-038:** cerrar con “una respuesta = una demostración”.

Frase clave

“El éxito en Prolog no es un resultado; es una prueba matemática.”

Checkpoint

Preguntar: “¿Qué pasa si en `abuelo(X,Z) :- progenitor(X,Y), progenitor(Y,Z).` invierto el orden de los goals?” → (corrección: en la semántica pura es equivalente; en la operacional cambia la eficiencia).

BLOQUE 3 — Backtracking (60–85 min) · F-039 a F-054

Objetivo: ver el motor “en vivo” con el backtracking explorando alternativas.

Guion

- **60–63 min · F-039, F-040:** definir backtracking y choice point. Analogía: mochila de “camino no tomados”.
- **63–68 min · F-041 a F-043:** live-coding con `color/1` y `almuerzo/2` en SWISH. Mostrar `;` para siguiente solución.
- **68–73 min · F-044 a F-046:** árbol de búsqueda bifurcado en pizarrón + explicación del trail.

- **73–78 min · F-047 a F-049:** múltiples soluciones + cuándo NO hay backtracking (determinismo).
- **78–82 min · F-050 a F-052:** árbol no trivial `conoce/2` + el peligro de ciclos infinitos.
- **82–85 min · F-053, F-054:** comparación con streams + cheatsheet.

Live-coding (SWISH)

```
color(rojo). color(verde). color(azul).
?- color(X), write(X), nl, fail.
?- color(X), color(Y), X \= Y.
```

Checkpoint

“¿Qué pasa si pongo `!` al final del cuerpo de `color/1`?” — adelantar el próximo bloque.

Pausa socrática obligatoria (min 80, ~2 min)

Antes de pasar a corte, **parar y preguntar al aula:**

*“Tengo `color(rojo). color(verde). color(azul). ?- color(X), X \= rojo, write(X).`
¿Cuántas veces backtrakea Prolog antes de imprimir algo? ¿Y qué imprime?”*

Dejar 30s de silencio. Levantar manos. Responder tú mismo si no sale: **backtrakea 1 vez** (rechaza `rojo`), imprime `verde`. Si alguien pregunta “¿no imprime `azul` también?” → decir “solo si fuerzo con `;` o `fail`” y enlazar con B4.

Esta pausa es el **checkpoint de salud cognitiva del cohort**: si nadie responde → repetir árbol SLD antes de seguir.

BLOQUE 4 — Corte (!) (85–105 min) · F-055 a F-066

Objetivo: entender el trade-off eficiencia/declaratividad.

Guion

- **85–88 min · F-055, F-056:** definir `!` como compromiso irrevocable.
- **88–93 min · F-057, F-058:** ejemplo `max/3` con corte verde vs. sin corte.
- **93–98 min · F-059, F-060:** corte rojo con `abs/2` — **importante:** mostrar qué pasa sin el `!`.
- **98–102 min · F-061 a F-063:** la barrera del corte en el árbol + implementación de `not/1`.
- **102–105 min · F-064 a F-066:** trampas + reglas de uso.

Frase clave

"Si al quitar el ! tu programa sigue bien, era corte verde. Si rompe, era rojo — y documentaló."

Checkpoint

"¿Preferís ! o (-> ;) para esto?" — pedir votación.

BLOQUE 5 — Negación por falla (105–120 min) · F-067 a F-075

Objetivo: distinguir falla de falsedad (CWA).

Guion

- **105–108 min · F-067, F-068:** definir $\backslash +$ y CWA.
- **108–113 min · F-069 a F-071:** la trampa del mundo abierto + variables libres.
- **113–117 min · F-072, F-073:** $\text{not}/1$ deprecado; $\text{dif}/2$ como alternativa correcta.
- **117–120 min · F-074, F-075:** ejemplo $\text{terrestre}/1$ + resumen.

Checkpoint

"¿Por qué $?- \backslash + X = 1, X = 2.$ falla pero $?- X = 2, \backslash + X = 1.$ tiene éxito?" — descuento directo, este punto **SIEMPRE** confunde.



DESCANSO (120–130 min)

10 minutos reales. No negociables. Después de 2 horas de Prolog, el cerebro necesita salir.

Yo aprovecho para: - Responder preguntas individuales - Chequear que el SWISH sigue andando - Limpiar el pizarrón

BLOQUE 6 — Ejercicio colaborativo (130–150 min) · F-076 a F-083

Objetivo: consolidación activa.

Guion

- **130–135 min · F-076, F-077:** explicar las reglas (pares, 5 min por ejercicio, puesta en común).
- **135–140 min · F-078:** trabajo en pares sobre `comparten_destino`.
- **140–141 min · F-079:** puesta en común (1 pareja pasa al pizarrón).
- **141–146 min · F-080, F-081:** trabajo + puesta en común `\+` con orden.
- **146–150 min · F-082, F-083:** reescribir `signo/2` con `!` y `(-> ;)`.

Nota

Si los alumnos se traban **más de 7 min** en cualquier ejercicio, avanzar y volver al final.

BLOQUE 7 — Aritmética (150–170 min) · F-084 a F-096

Objetivo: evitar el bug clásico de tratar expresiones como números.

Guion

- **150–153 min · F-084, F-085:** el “momento incómodo” — `?- X = 2+3.` no es 5.
- **153–157 min · F-086:** errores comunes con `is/2`. Hacerlos en vivo para que duela.
- **157–160 min · F-087, F-088:** tabla de operadores + trampa del `=<`.
- **160–163 min · F-089:** demos relámpago.
- **163–167 min · F-090, F-091:** factorial correcto vs. anti-patrón.
- **167–170 min · F-092 a F-096:** fibonacci + acumulador + CLP(FD) como preview.

Live-coding (SWISH)

```
?- X = 2+3, Y is X.  
?- X is 2+3.  
factorial(0,1). factorial(N,F) :- N>0, N1 is N-1, factorial(N1,F1), F is N*F1.  
?- factorial(10, F).
```

Checkpoint

“¿Cuál es la diferencia entre `==`, `is`, `=`?” — escribirlas en el pizarrón con 1 ejemplo de cada una.

BLOQUE 8 — Listas (170–195 min) · F-097 a F-112

Objetivo: dominar [H|T] y las primitivas.

Guion

- **170–175 min · F-097 a F-099:** importancia + notación + estructura interna (dibujar la cons list).
- **175–180 min · F-100, F-101:** pattern matching + vacía/no vacía.
- **180–185 min · F-102, F-103:** length/2 y member/2 — escribir ambas en vivo en pizarra.
- **185–190 min · F-104, F-105:** append/3 y su reversibilidad — **dedicar 5 min completos**. Este es el momento “wow” de la clase.
- **190–193 min · F-106 a F-108:** last, nth, sort, msort, pares clave-valor.
- **193–195 min · F-109 a F-112:** listas anidadas + between/3, forall/2, construcción de listas.

Live-coding (SWISH)

```
?- append([1,2],[3,4],R).  
?- append(A,B,[1,2,3]).           % reversible  
?- member(X, [a,b,c]).  
?- length(L, 3).                   % generar lista de 3 variables
```

Frase clave

“append/3 es la demostración de que relacionar > calcular.”

BLOQUE 9 — Recursión avanzada (195–220 min) · F-113 a F-126

Objetivo: acumuladores, LCO, debugging.

Guion

- **195–197 min · F-113:** patrón base + recursivo.
- **197–202 min · F-114, F-115:** suma/2 ingenuo + problema de stack. Dibujar el stack crecer.
- **202–207 min · F-116, F-117:** suma con acumulador + explicación de LCO.
- **207–211 min · F-118:** reverse/2 ingenuo $O(n^2)$ vs. acumulador $O(n)$. **Comparación de tiempos** en SWISH con lista de 10.000.
- **211–215 min · F-119, F-120:** map/filter expresados en Prolog.

- **215–218 min · F-121, F-122:** `maplist/2-3`, `foldl/4` — HOF built-in.
- **218–219 min · F-123:** evitar ciclos con visitados.
- **219–220 min · F-124 a F-126:** árboles + `trace` + resumen.

Live-coding (SWISH)

```
?- numlist(1, 10000, L), time(reverse_naive(L, R)). % Lento
?- numlist(1, 10000, L), time(reverse(L, R)).      % rápido
?- trace, suma([1,2,3], S).
```

BLOQUE 10 — Meta-predicados (220–240 min NO — ojo, 200–220 min) · F-127 a F-136

Nota de tiempos: B10 va en la ventana **200–220 min** en el diseño. La minuta ajusta ritmo si B9 se pasa.

Guion

- **200–203 min · F-127, F-128:** `findall/3` — el workhorse.
- **203–207 min · F-129 a F-132:** `bagof/3`, `setof/3` + agrupación.
- **207–210 min · F-133:** `call/N`.
- **210–215 min · F-134, F-135:** promedio de edades + `aggregate_all/3`.
- **215–220 min · F-136:** resumen de meta-predicados.

Flexibilidad: si voy justo de tiempo → saltar F-131 (agrupación con variable libre) y profundizar en TP.

BLOQUE 11 — Aplicaciones (220–240 min) · F-137 a F-149

Objetivo: mostrar que Prolog sigue vivo.

Guion

- **220–222 min · F-137:** valor de Prolog.
- **222–225 min · F-138:** coloreo de mapas. **Live-coding.**
- **225–228 min · F-139:** sudoku en 4 líneas — **mostrar** el código, no explicar el detalle.
- **228–232 min · F-140, F-141:** vuelos + misma base, múltiples consultas.

- **232–234 min · F-142:** N-reinas (CLP(FD)).
- **234–236 min · F-143:** parser DCG — **1 minuto máximo.**
- **236–239 min · F-144 a F-148:** diagnóstico, planificación, sistema experto, grafo de conocimiento, restricciones laborales.
- **239–240 min · F-149:** ¿qué tienen en común? → la base + múltiples consultas + backtracking.

Si me quedo sin tiempo: mostrar solo F-138, F-140, F-142, F-147.

BLOQUE 12 — Prolog en 2026 (240–250 min — sobre tiempo) · F-150 a F-154

Este bloque es compresible a 5 min si es necesario.

Guion condensado

- **240–242 min · F-150:** ¿vivo?
 - **242–244 min · F-151:** Datalog (Google, Meta).
 - **244–247 min · F-152:** neuro-simbólico (LLM + Prolog).
 - **247–249 min · F-153:** implementaciones modernas.
 - **249–250 min · F-154:** por qué lo estudiamos.
-

BLOQUE 13 — Cierre (250–260 min) · F-155 a F-158

Guion

- **250–252 min · F-155:** síntesis en 5 puntos.
- **252–255 min · F-156:** consigna del TP.
- **255–258 min · F-157:** pregunta de salida — **escribir en post-it** y dejar en la puerta.
- **258–260 min · F-158:** agradecimiento + próxima clase (OO con TypeScript).

Frase de cierre

“Prolog no es el lenguaje que van a usar todos los días. Es el lenguaje que va a cambiar cómo piensan sobre los que sí usan.”

Material de apoyo

- **PDFs en** `ingesta/txt/` : Sebesta cap. 16, Gabbrielli cap. logic programming, Louden cap. 4.
 - **SWISH preparado con:** familia.pl, colores.pl, vuelos.pl, nreinas.pl.
 - **Filminas:** `filminas.md` + publicadas en Google Slides (ver `slides/slides-url.txt` tras cierre).
 - **Guía de estudio:** `guia-estudio.md` (lectura post-clase).
 - **Guía del profesor:** `guiaprofesor.md` (si alguien reemplaza).
-

Plan de contingencia (si me atraso)

Atraso	Acción
+10 min hasta B5	Comprimir B6 ejercicio (10 min vs 20)
+20 min hasta B7	Saltear F-131 + F-143 + F-148
+30 min hasta B10	Entregar B11 como "miren estas aplicaciones" en 5 min, sin demo
Cualquier atraso al final	B12 a 3 min (solo F-150, F-152, F-154)

Lo NO sacrificable: B1 (unificación), B2 (SLD), B3 (backtracking), B8 (listas), B9 (acumulador), cierre.

Minuta — Paradigmas y Lenguajes de Programación 2026 — UNTdF — 2026-04-21 Trazabilidad:
`diseño.md` v2026-04-21 · `filminas.md` 158 slides · duración 240 min